

REAL-TIME FACIAL EXPRESSION DETECTION USING PYTHON AND DEEP LEARNING

Project Report

Submitted by: Tanvi Jaiswal

Registration No: 26MIB10015

Table of Contents

1. Abstract

This project presents a real-time facial expression detection system that identifies human emotions from webcam video using computer vision and deep learning. The system uses OpenCV for face detection and a Convolutional Neural Network (CNN), trained on the FER-2013 dataset, to classify each detected face into one of seven emotion categories: Angry, Disgust, Fear, Happy, Sad, Surprise, and Neutral. The predicted emotion and its confidence score are overlaid live on the video feed. The system is implemented in Python and is designed to run locally in VS Code, with clear separation between the model-training pipeline and the real-time inference application.

2. Introduction

Facial expressions are one of the most natural and immediate ways humans communicate emotion. Automated facial expression recognition (FER) has applications across human-computer interaction, mental health monitoring, driver fatigue/attention detection, customer sentiment analysis, and adaptive learning systems. Advances in deep learning, particularly Convolutional Neural Networks, have significantly improved the accuracy of automated emotion recognition compared to traditional handcrafted-feature approaches.

This project builds an end-to-end pipeline: a training script that learns to classify facial expressions from a labeled image dataset, and a real-time application that uses a live webcam feed to detect faces and classify their expressions on the fly.

3. Objectives

- Detect human faces in real time from a live webcam feed.
- Classify each detected face into one of seven standard emotion categories.
- Train a CNN-based deep learning model on a public benchmark dataset (FER-2013).
- Display live predictions with bounding boxes, labels, and confidence scores.
- Build a project structure that is easy to set up, run, and extend in VS Code.

4. Literature Review / Related Work

Early facial expression recognition systems relied on handcrafted features such as Local Binary Patterns (LBP), Histogram of Oriented Gradients (HOG), and Facial Action Coding System (FACS) rules, combined with classical classifiers like SVMs. These approaches were sensitive to lighting, pose, and occlusion. The introduction of deep Convolutional Neural Networks shifted the field toward learned, hierarchical feature representations that generalize better across varied conditions.

The FER-2013 dataset, introduced in a 2013 Kaggle competition, remains a widely used benchmark for emotion classification research. It contains roughly 35,000 grayscale 48x48 pixel face images labeled with seven emotion classes, making it a practical starting point for CNN-based FER systems despite known label noise and class imbalance (particularly for the 'Disgust' category).

5. System Requirements

5.1 Hardware Requirements

- A computer with a webcam (built-in or USB).
- Minimum 8 GB RAM (16 GB recommended for training).
- GPU (optional but recommended) for faster model training.

5.2 Software Requirements

- Python 3.9 – 3.11
- Visual Studio Code with the Python extension
- TensorFlow / Keras
- OpenCV (opencv-python)
- NumPy, Pandas, scikit-learn

6. Methodology

6.1 Dataset

The model is trained on the FER-2013 dataset, which provides 48x48 grayscale face images labeled with one of seven emotions, pre-split into Training, PublicTest (validation), and PrivateTest (test) subsets.

6.2 Face Detection

Faces are detected in each video frame using OpenCV's Haar Cascade frontal-face classifier, a fast and lightweight detector well suited to real-time applications on CPU.

6.3 Preprocessing

- Convert each frame to grayscale.
- Crop the detected face region.
- Resize the face to 48x48 pixels to match model input.
- Normalize pixel values to the [0, 1] range.

6.4 Model Architecture

The classifier is a sequential CNN composed of three convolutional blocks (with Batch Normalization, ReLU activations, Max Pooling, and Dropout for regularization), followed by a fully connected classification head with a final softmax layer producing probabilities over the seven emotion classes.

Layer	Configuration
Conv Block 1	2x Conv2D(64) + BatchNorm + MaxPool + Dropout(0.25)
Conv Block 2	2x Conv2D(128) + BatchNorm + MaxPool + Dropout(0.25)
Conv Block 3	Conv2D(256) + BatchNorm + MaxPool + Dropout(0.25)
Dense Head	Flatten -> Dense(256) + BatchNorm + Dropout(0.5)
Output	Dense(7, softmax)

6.5 Training Configuration

Parameter	Value
Optimizer	Adam
Loss Function	Categorical Crossentropy
Batch Size	64
Epochs	Up to 60 (with early stopping)
Data Augmentation	Rotation, zoom, shift, horizontal flip
Callbacks	ModelCheckpoint, EarlyStopping, ReduceLROnPlateau

6.6 Real-Time Inference Pipeline

- Capture frames continuously from the webcam using OpenCV's VideoCapture.
- Detect faces in each frame with the Haar Cascade classifier.

- Preprocess each detected face and pass it through the trained CNN.
- Draw a bounding box and the predicted emotion label with confidence on the frame.
- Display the annotated frame in a live window until the user presses 'q'.

7. System Architecture

The system follows a two-stage pipeline architecture: an offline training stage that produces a saved model file, and an online inference stage that loads this model to perform live predictions.

- Stage 1 (Offline): fer2013.csv -> Data Loading -> Preprocessing -> CNN Training -> emotion_model.h5
- Stage 2 (Online): Webcam Feed -> Face Detection -> Face Preprocessing -> CNN Inference -> Annotated Display

8. Implementation

The project is implemented in Python and organized into two main scripts. 'train_model.py' loads and preprocesses the FER-2013 dataset, builds and trains the CNN, and saves the best-performing model to disk. 'real_time_detection.py' loads this trained model, opens the webcam feed, detects faces frame-by-frame, and displays live emotion predictions. The two scripts are decoupled so the model can be retrained or swapped independently of the detection application.

8.1 Key Libraries Used

Library	Purpose
TensorFlow / Keras	Building, training, and running the CNN model
OpenCV (cv2)	Webcam capture, face detection, drawing overlays
NumPy	Array and image data manipulation
Pandas	Loading and parsing the FER-2013 CSV dataset
scikit-learn	Utility functions for data splitting/evaluation

9. Results

When trained for a sufficient number of epochs with data augmentation, CNN models of this architecture typically achieve test accuracy in the range of 60-70% on the FER-2013 PrivateTest split, consistent with published benchmarks for models of comparable depth. In real-time testing, the system reliably detects faces under normal indoor lighting and correctly identifies clearly expressed emotions such as Happy and Surprise with high confidence; visually similar classes such as Fear and Surprise, or Sad and Neutral, show more confusion, which is consistent with known FER-2013 class-overlap challenges.

[Insert screenshots of the live detection window and training accuracy/loss curves here.]

10. Challenges Faced

- Class imbalance in FER-2013, particularly the underrepresented 'Disgust' class.
- Sensitivity of Haar Cascade face detection to pose, occlusion, and lighting conditions.
- Balancing real-time frame rate against model inference latency.
- Avoiding overfitting given the relatively small, noisy-labeled dataset.

11. Conclusion

This project successfully demonstrates a complete real-time facial expression detection pipeline using Python, OpenCV, and a CNN built with TensorFlow/Keras. The system detects faces from a live webcam feed and classifies them into one of seven emotions with reasonable accuracy, showcasing a practical application of deep learning to computer vision and human-computer interaction problems.

12. Future Scope

- Replace the Haar Cascade detector with a more robust deep-learning face detector (e.g., MTCNN or MediaPipe).
- Use transfer learning from pretrained networks (e.g., MobileNetV2, ResNet) to improve accuracy.
- Extend the system to multi-face tracking and per-person emotion logging over time.
- Deploy the system as a web application using Flask/FastAPI for browser-based access.
- Fine-tune on a custom, more diverse dataset to reduce bias and improve robustness.

13. References

1. Goodfellow, I. J., et al. 'Challenges in Representation Learning: A Report on Three Machine Learning Contests.' (FER-2013 dataset origin), 2013.
2. Bradski, G. 'The OpenCV Library.' Dr. Dobb's Journal of Software Tools, 2000.
3. Chollet, F. et al. Keras. <https://keras.io>, 2015.
4. Kaggle FER-2013 Dataset: <https://www.kaggle.com/datasets/msambare/fer2013>